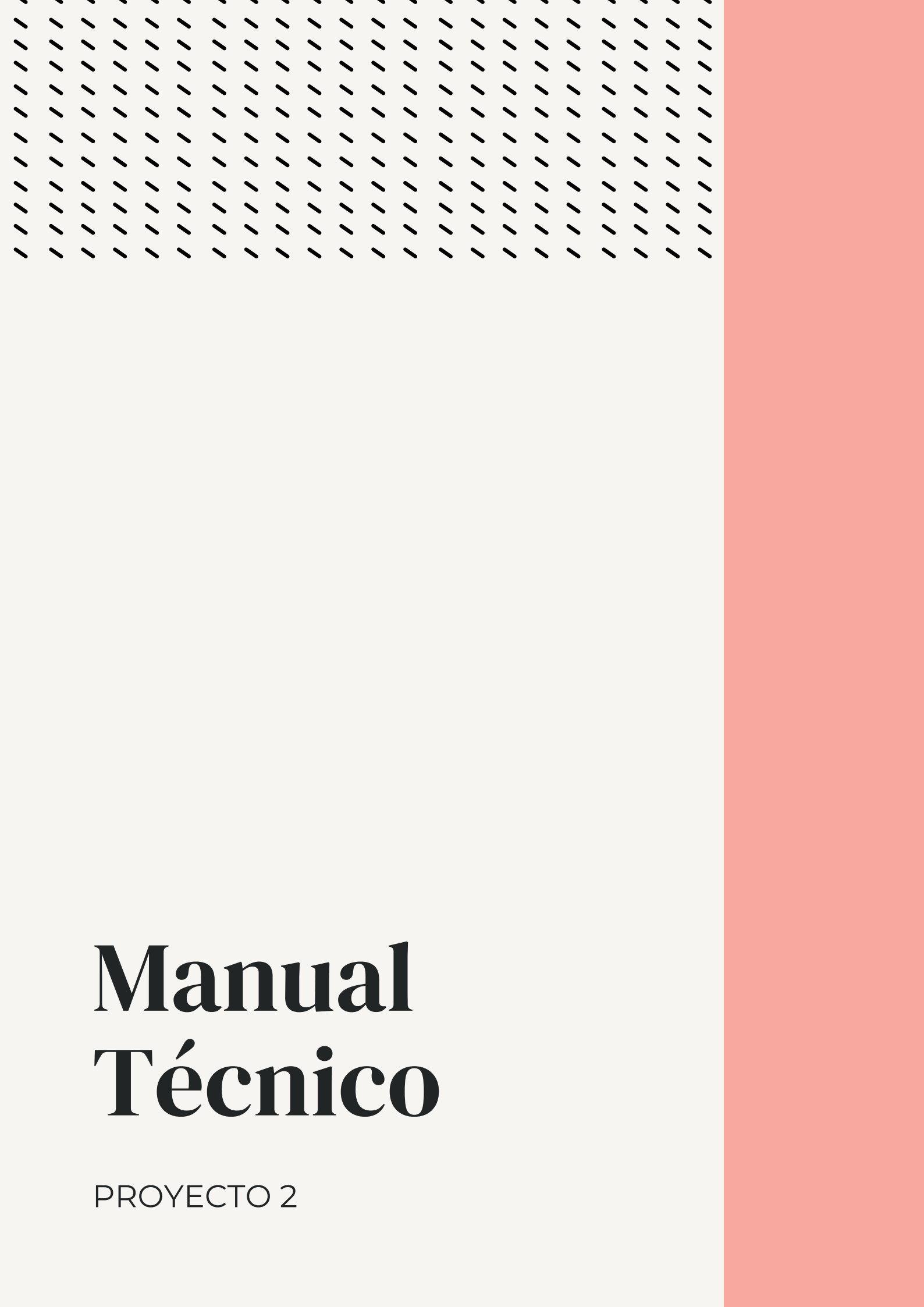




Manual Técnico

PROYECTO 2



Índice

02	Paradigma de Programación
03	Convenciones de Nomenclatura
04	Clases Principales
13	Métodos Principales
25	Scanner y Parser
28	Requerimientos o Funcionalidades Específicas
29	Interfaces Principales



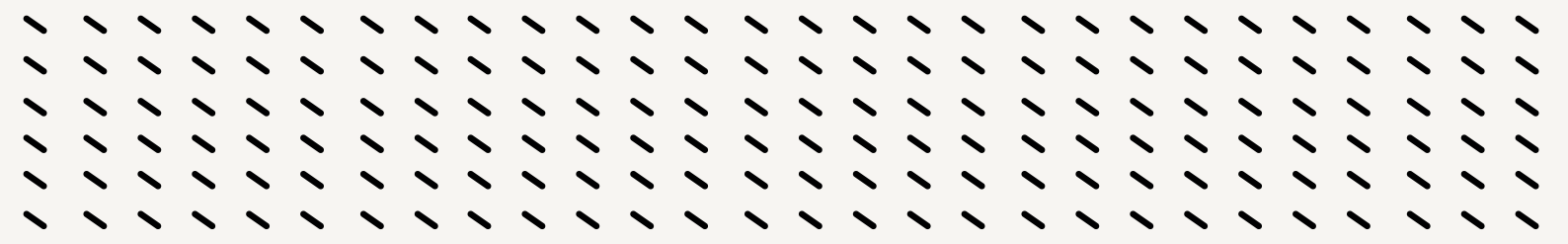
Paradigma de Programación

El proyecto utiliza un enfoque multi-paradigma, predominando la programación orientada a objetos en el backend, combinada con programación funcional y basada en eventos en el frontend, además de un estilo imperativo para el flujo de ejecución del intérprete.

Un ejemplo claro, es el manejo del documento que se analiza, este documento se maneja como un conjunto de objetos con elementos y propiedades.

También se utiliza el encapsulamiento al crear atributos privados y métodos públicos para acceder a estos.

Se utiliza el lenguaje Javascript para toda la programación del software y Jison para analizar el código ingresado por el usuario.



Convenciones de Nomenclatura

VARIABLES:

- Se declararon variables de tipo string para almacenar el contenido del código fuente ingresado por el usuario desde el editor, para manejar la salida de la consola y los reportes en formato HTML.
- Se declararon variables tipo array para almacenar listas de errores, archivos cargados y resultados del análisis.
- Se utilizaron variables tipo boolean para validar el estado de ejecución del análisis.
- Se emplearon variables tipo object para representar estructuras como el AST y los datos de respuesta del servidor.

ESTRUCTURAS:

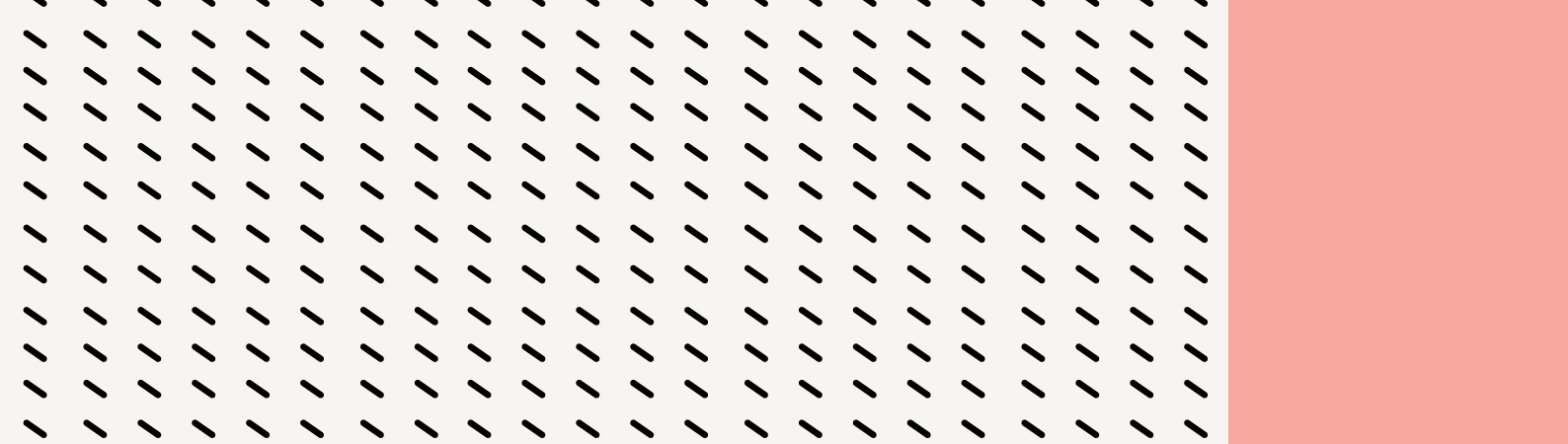
- Se utilizó la estructura tipo Array para almacenar la lista de errores generados durante el análisis.
- Se emplearon estructuras tipo Object (JSON) para representar nodos del AST y la información enviada entre frontend y backend.
- Se utilizó la estructura Map para manejar datos como reportes y configuraciones internas.
- Se implementó una estructura tipo lista dinámica para manejar múltiples archivos cargados en el editor.
- Se utilizó el AST como estructura jerárquica principal para representar el programa analizado.

CLASES:

- Se crearon clases como Consola para almacenar y gestionar la salida del programa interpretado.
- Se implementó la clase Errores para registrar, almacenar y generar reportes de errores en formato HTML.
- Se utilizó el parser generado como componente para construir el AST.
- Se implementó el Interpretador como módulo encargado de recorrer el AST y ejecutar las instrucciones.
- En el frontend se utilizaron componentes como App, Layout, Barra, Editor, Consola y Archivos para estructurar la interfaz gráfica.

MÉTODOS:

- Se implementaron métodos para analizar el código fuente y generar el AST.
- Se desarrollaron métodos para interpretar el AST y ejecutar las instrucciones del lenguaje.
- Se crearon métodos para agregar, limpiar y obtener errores.
- Se implementaron métodos para generar reportes en HTML y exportarlos a archivos.
- Se desarrollaron métodos para manejar archivos en el frontend.
- Se crearon métodos para enviar peticiones HTTP al backend y procesar la respuesta.



Clases Principales

Clases de Instrucciones

Las clases de instrucciones son las que ejecutan las diferentes instrucciones que contienen los nodos generados por el parser del programa. Estas clases crean, actualizan, eliminan o diferentes variables, estructuras o salidas.

```
Instrucciones.js M X
Backend > Instrucciones > Instrucciones.js > ...
1  const Entorno = require("../Instrucciones/Entorno");
2  const Tipos = require("../Instrucciones/Tipos");
3  const Consola = require("../Reports/Consola");
4  const Errores = require("../Reports/Errores");
5
6  class Declaracion {
7      constructor(id, tipo, valor) {
8          this.id = id;
9          this.tipo = tipo;
10         this.valor = valor;
11     }
12     evaluar(entorno) {
13         if (entorno.existeLocal(this.id)) {
14             Errores.agregar("Semántico", `La variable ${this.id} ya ha sido declarada.`);
15             return null;
16         }
17         let valorEvaluado;
18         if (this.valor) {
19             const resultado = this.valor.evaluar(entorno);
20             valorEvaluado = resultado.valor;
21         } else {
22             switch (this.tipo) {
23                 case 'int':
24                     valorEvaluado = 0;
25                     break;
26                 case 'float64':
27                     valorEvaluado = 0.0;
28                     break;
29                 case 'bool':
30                     valorEvaluado = false;
31                     break;
32                 case 'string':
```

Clase de Operaciones Aritméticas

Las clases de operaciones aritméticas son las encargadas de calcular las sumas, restas, multiplicaciones, divisiones y modulos de las instrucciones que se generan en los nodos al analizar el lenguaje.

```
Aritmetica.js X
Backend > Aritmetica > Aritmetica.js > ...
1  class Aritmetica {
2      constructor(izquierda, operador, derecha) {
3          this.izquierda = izquierda;
4          this.operador = operador;
5          this.derecha = derecha;
6      }
7
8      evaluar(entorno) {
9          const izq = this.izquierda.evaluar(entorno);
10         const der = this.derecha.evaluar(entorno);
11         if (!izq || !der) return null;
12         return this.resolverOperacion(izq, der, this.operador);
13     }
14
15     resolverOperacion(izq, der, op) {
16         const t1 = izq.tipo;
17         const t2 = der.tipo;
18         //console.log("IZQ:", izq);
19         //console.log("DER:", der);
20         /* CONVERSIONES */
21         const toNumber = (val, tipo) => {
22             switch (tipo) {
23                 case "int": return val;
24                 case "float64": return val;
25                 case "bool": return val ? 1 : 0;
26                 case "rune": return val.charCodeAt(0);
27             }
28         };
29         /* SUMA */
30         if (op === "+") {
31             if (t1 === "string" || t2 === "string") {
32                 return {
```

Clase de Operaciones Lógicas

Las clases de operaciones lógicas son las encargadas de calcular las comparaciones y operaciones lógicas de las instrucciones que se generan en los nodos al analizar el lenguaje.

```
Comparacion.js M X
Backend > Logica > Comparacion.js > Comparacion
1  class Comparacion {
2      constructor(izquierda, operador, derecha) {
3          this.izquierda = izquierda;
4          this.operador = operador;
5          this.derecha = derecha;
6      }
7
8      evaluar(entorno) {
9          const izq = this.izquierda.evaluar(entorno);
10         const der = this.derecha.evaluar(entorno);
11         if (!izq || !der) {
12             return { tipo: 'boolean', valor: false };
13         }
14         const valIzq = izq.valor;
15         const valDer = der.valor;
16         let resultado;
17         switch (this.operador) {
18             case "==": resultado = valIzq == valDer; break;
19             case "!=": resultado = valIzq != valDer; break;
20             case ">": resultado = valIzq > valDer; break;
21             case "<": resultado = valIzq < valDer; break;
22             case ">=": resultado = valIzq >= valDer; break;
23             case "<=": resultado = valIzq <= valDer; break;
24             default:
25                 throw new Error("Operador desconocido");
26         }
27         return {
28             tipo: 'boolean',
29             valor: resultado
30         };
31     }
32 }
```

Clase Consola

En la clase Consola se almacenan todas las salidas generadas por la instrucción Imprimir, que luego se mostraran en la consola del frontend.

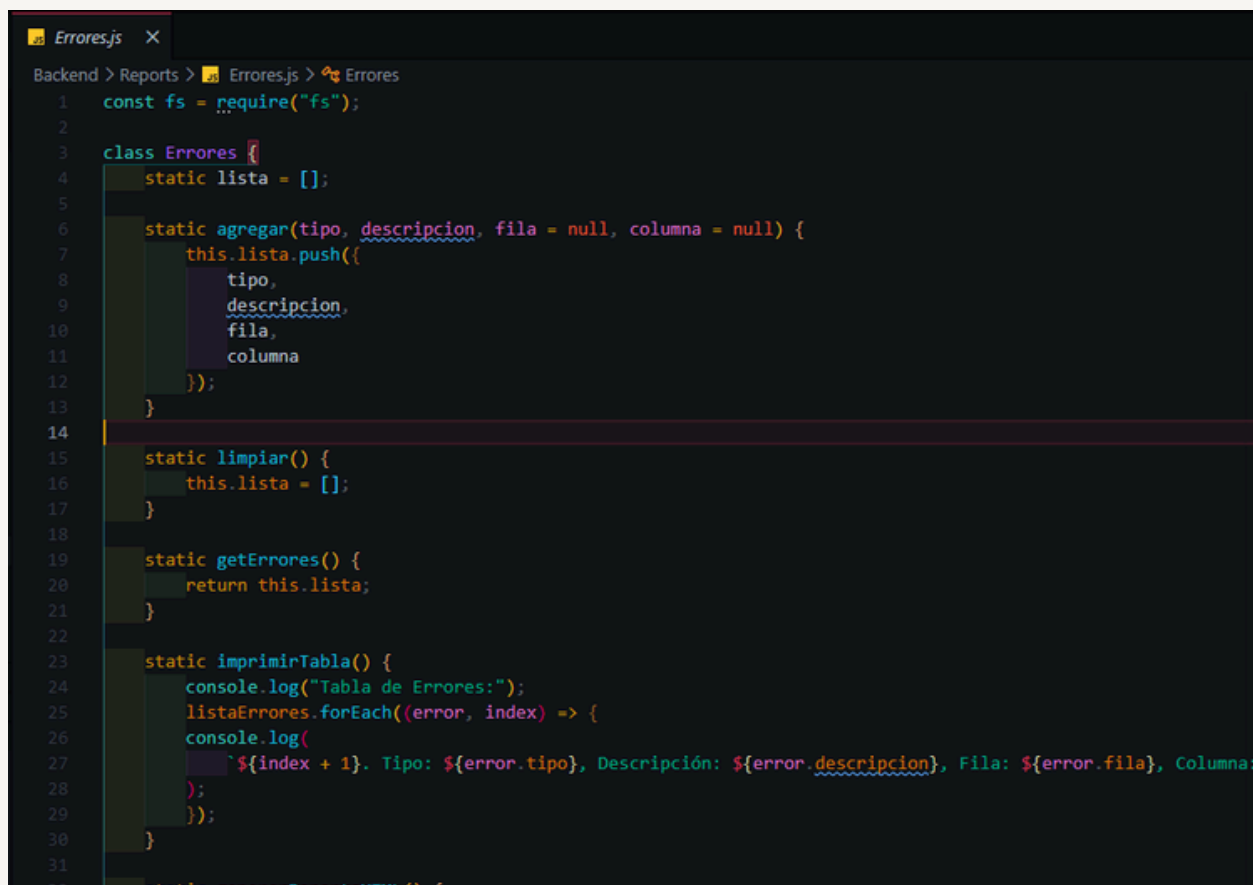
Utiliza estructuras tipo lista para almacenar todas las salidas y métodos para limpiar la consola.

```
JS Consola.js X
Backend > Reports > JS Consola.js > ...
1  class Consola {
2      static salida = [];
3
4      static log(...valores) {
5          this.salida.push(valores.join(" "));
6      }
7
8      static limpiar() {
9          this.salida = [];
10     }
11
12     static getSalida() {
13         return this.salida.join("\n");
14     }
15 }
16
17 module.exports = Consola;
```

Clase Errores

En la clase Consola se almacenan todas los errores generados por el parser, el interpretador o durante la ejecución.

Utiliza estructuras tipo lista para almacenar todos los errores y métodos para generar un reporte HTML.



```
1  const fs = require("fs");
2
3  class Errores {
4      static lista = [];
5
6      static agregar(tipo, descripcion, fila = null, columna = null) {
7          this.lista.push({
8              tipo,
9              descripcion,
10             fila,
11             columna
12         });
13     }
14
15     static limpiar() {
16         this.lista = [];
17     }
18
19     static getErrores() {
20         return this.lista;
21     }
22
23     static imprimirTabla() {
24         console.log("Tabla de Errores:");
25         listaErrores.forEach((error, index) => {
26             console.log(
27                 `${index + 1}. Tipo: ${error.tipo}, Descripción: ${error.descripcion}, Fila: ${error.fila}, Columna:
28             );
29         });
30     }
31
32     static generarReporteHTML() {
```

Clase Entorno

En la clase Entorno se declaran, asignan, y se muestran las diferentes variables, estructuras de datos y funciones definidas por el lenguaje analizado.

Utiliza estructuras Map para almacenar variables, estructuras de datos y funciones y permite comprobar la existencia de estas.

```
Entorno.js
Backend > Instrucciones > Entorno.js > Consola
1  const Consola = require("../Reports/Consola");
2  const Errores = require("../Reports/Errores");
3  class Entorno {
4      constructor(padre = null) {
5          this.tabla = new Map();
6          this.padre = padre;
7          this.erros = [];
8      }
9
10     declarar(id, valor) {
11         if (this.tabla.has(id)) {
12             this.erros.push(`Variable ya definida: ${id}`);
13             Errores.agregar("Semántico", "Variable ya definida: " + id);
14         } else {
15             this.tabla.set(id, valor);
16         }
17     }
18
19     obtener(id) {
20         if (typeof id === "object") {
21             if (id.tipo === "Identificador") {
22                 id = id.valor;
23             } else if (id.id) {
24                 id = id.id;
25             }
26         }
27         const variable = this.tabla.get(id);
28         if (variable !== undefined) {
29             return variable;
30         }
31         if (this.padre) {
32             return this.padre.obtener(id);
33         }
34     }
35 }
```

Clases Valores

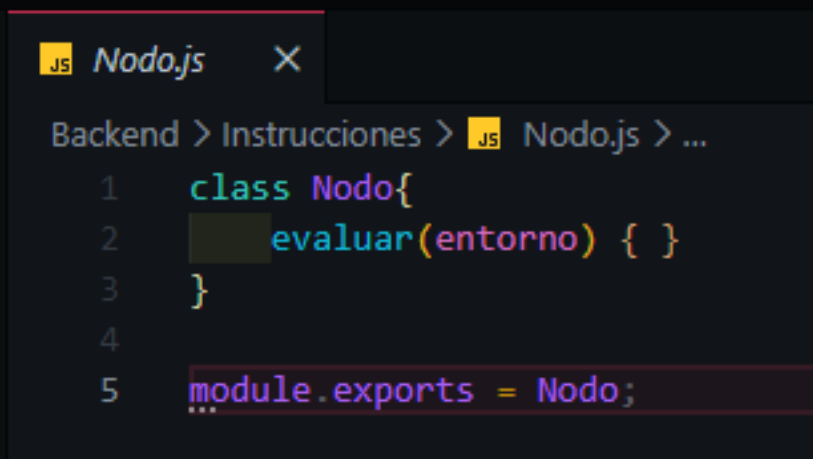
En la Clase valores se evalúan los diferentes tipos de valores que el lenguaje permite y se genera un objeto con el valor y el tipo del valor.

Se basa en la Clase Node para la estructura de evaluación de los valores.

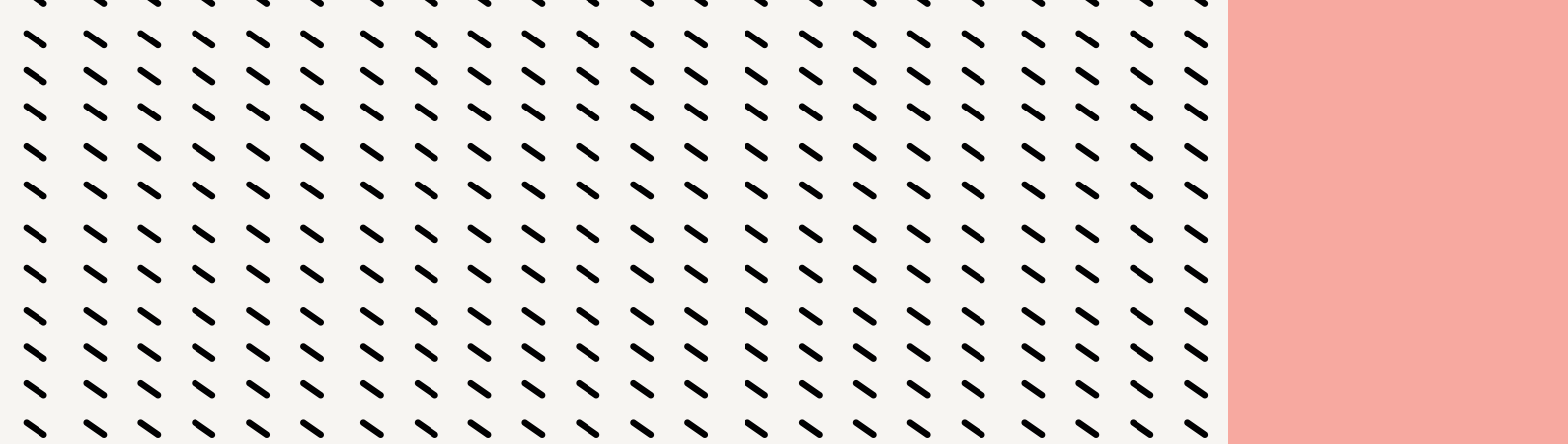
```
Valores.js
Backend > Instrucciones > Valores.js > string > constructor
1  const Nodo = require('./Nodo');
2  const Entorno = require("../Instrucciones/Entorno");
3
4  class rune extends Nodo{
5      constructor(valor) {
6          super();
7          this.valor = valor;
8      }
9      evaluar(entorno) {
10         return { tipo: 'rune', valor: this.valor};
11     }
12 }
13
14
15 class int extends Nodo {
16     constructor(valor) {
17         super();
18         this.valor = valor;
19     }
20     evaluar() {
21         return { tipo: 'int', valor: this.valor};
22     }
23 }
24
25 class float64 extends Nodo {
26     constructor(valor) {
27         super();
28         this.valor = valor;
29     }
30     evaluar() {
31         return { tipo: 'float64', valor: this.valor};
32     }
33 }
```

Clases Nodo

Sirve para dar estructura a algunas clases, agregándoles el método evaluar que se encarga de ejecutar las instrucciones correspondientes y devolver un valor.



```
JS  Nodo.js  X
Backend > Instrucciones > JS  Nodo.js > ...
1  class Nodo{
2    evaluar(entorno) { }
3  }
4
5  module.exports = Nodo;
```



Métodos Principales

Método interpretar()

El método interpretar se encuentra en el Interpretador, este método se encarga de iterar los nodos que se encuentran en el AST y enviarlos a interpretar.

Este método también se encarga de devolver, los datos almacenados, las salidas de consola y los errores generados por el lenguaje.

```
function interpretar(nodo) {  
  const entorno = new Entorno();  
  try {  
    if (Array.isArray(nodo)) {  
      nodo.forEach(n => {  
        const raiz = aNodo(n);  
        if (raiz) raiz.evaluar(entorno);  
      });  
    } else {  
      const raiz = aNodo(nodo);  
      if (raiz) raiz.evaluar(entorno);  
    }  
  } catch (e) {  
    Errores.agregar("Fatal", e.toString());  
  }  
  return {  
    entorno,  
    consola: Consola.getSalida(),  
    errores: Errores.getErrores()  
  };  
}  
  
module.exports = interpretar ;
```

Método evaluar()

Todas las clases de instrucciones cuentan con un método evaluar, en este método se ejecutan las instrucciones correspondientes por cada instrucción generada por el lenguaje.

Los métodos evaluar recibe una instancia del entorno y devuelven un objeto o un valor null para que la siguiente instrucción se ejecute.

```
evaluar(entorno) {  
  const izq = this.izquierda.evaluar(entorno);  
  const der = this.derecha.evaluar(entorno);  
  if (!izq || !der) {  
    return { tipo: 'boolean', valor: false };  
  }  
  const valIzq = izq.valor;  
  const valDer = der.valor;  
  let resultado;  
  switch (this.operador) {  
    case "==" : resultado = valIzq == valDer; break;  
    case "!=" : resultado = valIzq != valDer; break;  
    case ">"  : resultado = valIzq > valDer; break;  
    case "<"  : resultado = valIzq < valDer; break;  
    case ">=" : resultado = valIzq >= valDer; break;  
    case "<=" : resultado = valIzq <= valDer; break;  
    default :  
      throw new Error("Operador desconocido");  
  }  
  return {  
    tipo: 'boolean',  
    valor: resultado  
  };  
}
```

Método resolver()

Los métodos resolver se crearon para realizar las instrucciones que pueden generar un conflicto por el tipo de datos que generan.

Estos métodos reciben ciertos datos y devuelven un valor o un tipo de valor sin conflictos evitando errores internos.

```
resolverOperacion(izq, der, op) {
  const t1 = izq.tipo;
  const t2 = der.tipo;
  /* CONVERSIONES */
  const toNumber = (val, tipo) => {
    switch (tipo) {
      case "int": return val;
      case "float64": return val;
      case "bool": return val ? 1 : 0;
      case "rune": return val.charCodeAt(0);
    }
  };
  /* SUMA */
  if (op === "+") {
    if (t1 === "string" || t2 === "string") {
      return {
        valor: String(izq.valor) + String(der.valor),
        tipo: "string"
      };
    }
    if (t1 === "float64" || t2 === "float64") {
      return {
        valor: toNumber(izq.valor, t1) + toNumber(der.valor, t2),
        tipo: "float64"
      };
    }
    if (t1 === "bool" && t2 === "bool") {
      return {
        valor: izq.valor || der.valor,

```

Método del Entorno

Estos métodos se encargan de administrar y crear espacios en memoria para funciones, estructuras y variables generadas por el lenguaje analizado.

Algunos de los métodos son existe(), asignar(), obtenerTipo() y declarar().

```
existe(id) {
  let actual = this;
  while (actual != null) {
    if (actual.tabla.has(id)) return true;
    actual = actual.padre;
  }
  return false;
}

asignar(id, valor) {
  let actual = this;
  while (actual != null) {
    if (actual.tabla.has(id)) {
      actual.tabla.set(id, valor);
      //console.log("SET:", id, valor);
      return;
    }
    actual = actual.padre;
  }
  this.errores.push(`Variable no definida: ${id}`);
  Errores.agregar("Semántico", "Variable no definida: " + id);
}

function obtenerTipo(valor) {
  if (typeof valor === 'boolean') return 'bool';
  if (typeof valor === 'string') return 'string';
  if (typeof valor === 'number') {
    return Number.isInteger(valor) ? 'int' : 'float64';
  }
  return 'unknown';
}

module.exports = { obtenerTipo };

declarar(id, valor) {
  if (this.tabla.has(id)) {
    this.errores.push(`Variable ya definida: ${id}`);
    Errores.agregar("Semántico", "Variable ya definida: " + id);
  } else {
    this.tabla.set(id, valor);
  }
}
```

Método generarReporte()

En este método se genera el reporte HTML, y se utiliza en errores, símbolos y tokens.

```
static generarReporteHTML() {  
  let html = `  
    <html>  
    <head><title>Reporte de Errores</title></head>  
    <body>  
    <h1>Reporte de Errores</h1>  
    <table border="1">  
      <tr>  
        <th>No.</th>  
        <th>Tipo</th>  
        <th>Descripción</th>  
        <th>Fila</th>  
        <th>Columna</th>  
      </tr>  
    `;  
  this.lista.forEach((error, index) => {  
    html += `  
      <tr>  
        <td>${index + 1}</td>  
        <td>${error.tipo}</td>  
        <td>${error.descripcion}</td>  
        <td>${error.fila}</td>  
        <td>${error.columna}</td>  
      </tr>  
    `;  
  });  
  html += `</table></body></html>`;  
  return html;  
}
```

Método de Componentes()

Estos métodos contienen todos los objetos que se visualizaran en el fronten, como botones, areas de texto, etiquetas, etc

```
1  function Consola({ output }) {  
2    return (  
3      <div className="consola">  
4        <h4 className="nombreConsola"> Consola</h4>  
5        <div className="output">  
6          <pre>{output}</pre>  
7        </div>  
8      </div>  
9    );  
10  }  
11  export default Consola;
```

Método Layout()

La función Layout contiene todos los componentes y sus funciones y le da la estructura a la ventana del frontend.

```
function Layout({ contenidoEditor, setContenidoEditor, ejecutar, output, reporteHTML, archivos,
  subirArchivo, seleccionarArchivo, fileInputRef, guardarArchivo, crearArchivo, eliminarArchivo, archivoActivo })
  return (
    <div className="container">
      <Archivos
        archivos={archivos}
        archivoActivo={archivoActivo}
        subirArchivo={subirArchivo}
        seleccionarArchivo={seleccionarArchivo}
        crearArchivo={crearArchivo}
        eliminarArchivo={eliminarArchivo}
        fileInputRef={fileInputRef}
      />
      <div className="main">
        <Barra ejecutar={ejecutar} reporteHTML={reporteHTML} guardarArchivo={guardarArchivo}/>
        <Editor
          contenidoEditor={contenidoEditor}
          setContenidoEditor={setContenidoEditor}
        />
        <Consola output={output} />
      </div>
    </div>
  );
}
```

Método post()

En el método port el backend del proyecto recibe el código del lenguaje que va a analizar y devuelve las salidas y errores que se generaron con ese código.

Este método también envía el reporte html para que se muestre en una ventana dentro del frontend.

```
app.post("/analizar", (req, res) => {
  const { codigo } = req.body;
  try {
    Consola.limpiar();
    Errores.limpiar();
    const ast = parser.parse(codigo);
    const dot = generarAST(ast);
    fs.writeFileSync("ast.dot", dot);
    exec("dot -Tpdf ast.dot -o ast.pdf");
    const resultado = Interpretador(ast);
    const htmlErrores = Errores.generarReporteHTML();

    res.json({
      ok: true,
      consola: Consola.getSalida(),
      errores: Errores.getErrores(),
      htmlErrores
    });
  } catch (error) {
    Errores.agregar("Sintáctico", error.message);
    res.json({
      ok: false,
      consola: Consola.getSalida(),
      errores: Errores.getErrores(),
      htmlErrores: Errores.generarReporteHTML()
    });
  }
  Errores.crearReporteErrores();
});
```

Método App()

En este método el frontend envía el código a analizar y recibe las salidas y errores que genere este código.

En este método también se crean las acciones que se ejecutarán en el frontend como la subida de archivos y mostrar los diferentes reportes.

```
function App() {  
  const [contenidoEditor, setContenidoEditor] = useState("");  
  const [reporteHTML, setReporteHTML] = useState("");  
  const [archivos, setArchivos] = useState([]);  
  const [archivoActivo, setArchivoActivo] = useState(null);  
  const [output, setOutput] = useState("");  
  
  const fileInputRef = useRef(null);  
  const ejecutar = async () => {  
    try {  
      const res = await fetch("http://localhost:3000/analizar", {  
        method: "POST",  
        headers: {  
          "Content-Type": "application/json"  
        },  
        body: JSON.stringify({  
          codigo: contenidoEditor  
        })  
      });  
      const data = await res.json();  
      if (data.ok) {  
        let salida = data.consola;  
        if (data.errores.length > 0) {  
          salida += "\n\nErrores:\n";  
          data.errores.forEach(err => {  
            salida += `[${err.tipo}] ${err.descripcion\\n`;  
          });  
        }  
        setOutput(salida);  
        setReporteHTML(data.htmlErrores);  
      } else {  

```

Html del frontend

El HTML del proyecto crea la estructura de la ventana en la que se visualizara en frontend y los archivos de los que tomara estilos y el código a ejecutar, entre otras configuraciones.

```
Frontend > index.html > html > head > link
1  <!doctype html>
2  <html lang="en">
3
4  <head>
5    <meta charset="UTF-8" />
6    <link rel="icon" type="image/png" href="/Pompompurim.png" />
7    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
8    <title>Proyecto 2</title>
9  </head>
10
11 <body>
12   <div id="root"></div>
13   <script type="module" src="/src/main.jsx"></script>
14 </body>
15
16 </html>
```

CSS del Frontend

En este archivo se encuentran todas las configuraciones y estilos de los elementos que se mostraran en el frntend del proyecto.

```
.container { ...
}

.archivos {
  background: #252526;
  color: #ccc;
  padding: 12px;
  border-right: 1px solid #333;
  font-size: 14px;
}

.archivos ul {
  list-style: none;
  padding: 0;
  margin: 0;
}

.archivos h3 {
  font-size: 13px;
  color: #888;
  margin-bottom: 10px;
}

.archivos div {
  padding: 6px 8px;
  border-radius: 4px;
  cursor: pointer;
}

.archivos div:hover {
  background: #37373d;
}
```



Scanner y Parser

Parser

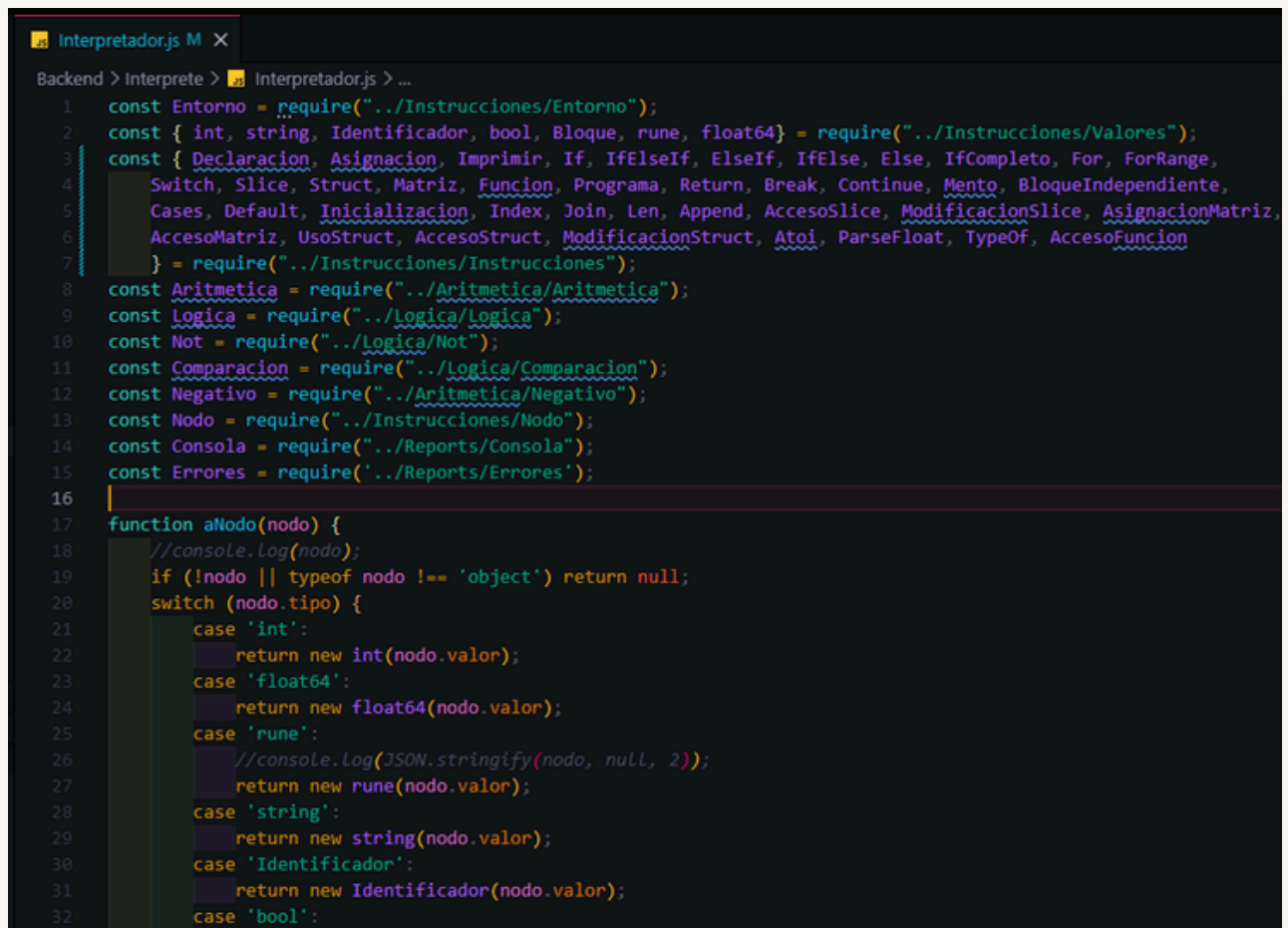
El parser del proyecto esta escrito en lenguaje jison. Jison tiene la particularidad de que se puede crear el lexer y el parser del lenguaje en el mismo archivo, por lo que en este mismo archivo se definen los tokens y el orden en el que se esperan.

Desde este parser se construye el AST que luego se envia al index del proyecto donde se grafica y se envia a interpretar las instrucciones.

```
parser.jison X
Backend > Parser > parser.jison
1  %{
2      const Token = require('../Reports/Tokens');
3      const Simbolo = require('../Reports/Simbolos');
4      const ErrorL = require('../Reports/Errores');
5      const TablaTokens = require('../Reports/TablaTokens');
6      const TablaSimbolos = require('../Reports/TablaSimbolos');
7      const Tipos = require('../Instrucciones/Tipos');
8
9      let ambito= "";
10     let contadorBloques=0;
11 }
12 /* Lexical grammar */
13 %lex
14 %%
15
16 \s+                { /* skip whitespace */ }
17
18 /* COMENTARIOS */
19 "/*".*              { /* comentario de una línea */ }
20 "/*"[^"]*"*/"       { /* comentario multi-línea */ }
21
22 /* TIPOS DE DATOS */
23 "int"                { const intToken = new Token("INT", yytext, yylineno, yylloc.first_column);
24                       TablaTokens.agregarToken(intToken);
25                       return 'INT';
26                       }
27 "float64"            { const float64Token = new Token("FLOAT", yytext, yylineno, yylloc.first_column);
28                       TablaTokens.agregarToken(float64Token);
29                       return 'FLOAT';
30                       }
31 "string"              { const stringToken = new Token("STRING", yytext, yylineno, yylloc.first_column);
32                       TablaTokens.agregarToken(stringToken);
```

Interpretador

El interpretador recibe los nodos que contiene el AST y los redirige a las instrucciones correspondientes separando los diferentes elementos del nodo.



```
Interpretador.js M X
Backend > Interprete > Interpretador.js > ...
1  const Entorno = require("../Instrucciones/Entorno");
2  const { int, string, Identificador, bool, Bloque, rune, float64 } = require("../Instrucciones/Valores");
3  const { Declaracion, Asignacion, Imprimir, If, IfElseIf, ElseIf, IfElse, Else, IfCompleto, For, ForRange,
4        Switch, Slice, Struct, Matriz, Funcion, Programa, Return, Break, Continue, Mento, BloqueIndependiente,
5        Cases, Default, Inicializacion, Index, Join, Len, Append, AccesoSlice, ModificacionSlice, AsignacionMatriz,
6        AccesoMatriz, UsoStruct, AccesoStruct, ModificacionStruct, Atoi, ParseFloat, TypeOf, AccesoFuncion
7        } = require("../Instrucciones/Instrucciones");
8  const Aritmetica = require("../Aritmetica/Aritmetica");
9  const Logica = require("../Logica/Logica");
10 const Not = require("../Logica/Not");
11 const Comparacion = require("../Logica/Comparacion");
12 const Negativo = require("../Aritmetica/Negativo");
13 const Nodo = require("../Instrucciones/Nodo");
14 const Consola = require("../Reports/Consola");
15 const Errores = require("../Reports/Errores");
16
17 function aNodo(nodo) {
18     //console.log(nodo);
19     if (!nodo || typeof nodo !== 'object') return null;
20     switch (nodo.tipo) {
21         case 'int':
22             return new int(nodo.valor);
23         case 'float64':
24             return new float64(nodo.valor);
25         case 'rune':
26             //console.log(JSON.stringify(nodo, null, 2));
27             return new rune(nodo.valor);
28         case 'string':
29             return new string(nodo.valor);
30         case 'Identificador':
31             return new Identificador(nodo.valor);
32         case 'bool':
```



Requerimientos o Funcionalidad Especifica

VISUAL ESTUDIO CODE:

Se utilizo este editor de código fuente por la familiaridad del entorno y el fácil manejo para la ejecución que ofrece para trabajar con el lenguaje de programación Javascript.

NAVEGADOR WEB:

Se utilizo un navegador web para visualizar la ejecución del frontend de la aplicación.

Interfaces Principales

ARCHIVOS: Permite administrar los archivos con las opciones “Nuevo Archivo” y “Subir Archivo”.

BARRA DE HERRAMIENTAS: Muestra las opciones de “Ejecutar” o “Guardar” el archivo y permite visualizar La Tabla de Simbolos, Tokens, Errores y el AST

EDITOR: Muestra el contenido del archivo .eli elegido.

CONSOLA: Muestra el estado de la ejecución del lenguaje, las salidas y los errores generados por el lenguaje.

